

UNIDAD 8:

ENTRENAMIENTO SUPERVISADO

CURSO

IA

```
01 import openai
02 from langchain import
03     ChatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = ChatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10
11     respuesta = modelo.invoke(
12         prompt
13     )
14     return respuesta.content
15
16 if __name__ == "__main__":
17     prompt = "Explicame qué
18     es la inteligencia artificial"
19     print(generar_respuesta(prompt))
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep.Argentina 786, Sunchales, Sta fe.



GabrielNMosso@GMail.com



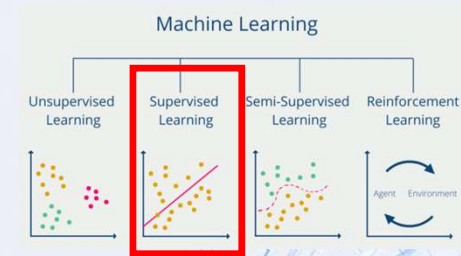
Tel: +54 351 3733383



ENTRENAMIENTO SUPERVISADO

El **entrenamiento supervisado** es una de las formas más comunes y fundamentales de enseñar a una inteligencia artificial.

Se basa en el uso de **datos etiquetados** o sea cada ejemplo que se le muestra al modelo ya viene con la **respuesta correcta**.



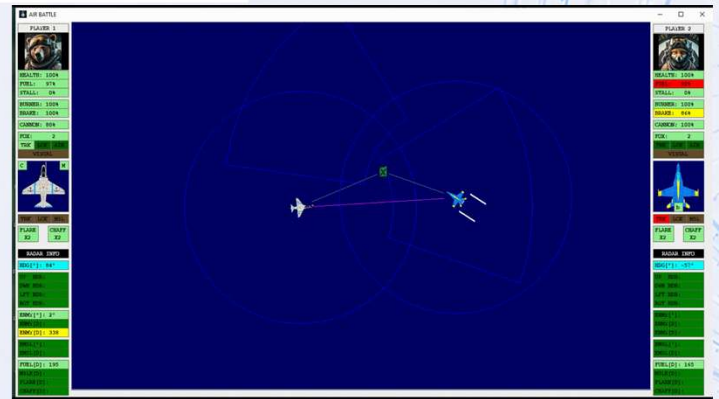
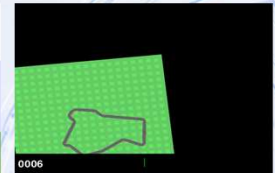
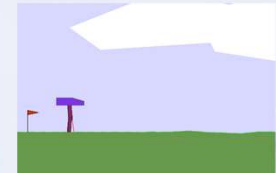
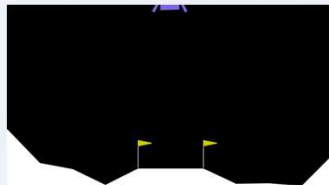
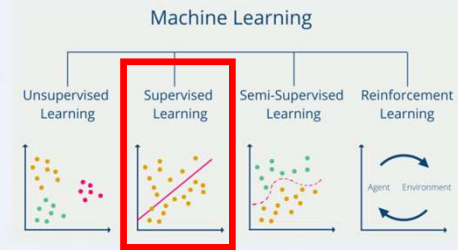
Aspecto	Descripción
Definición	Método de entrenamiento basado en datos etiquetados, donde cada entrada tiene una salida conocida.
Funcionamiento	1. Se entregan ejemplos con entradas y salidas correctas. 2. El modelo aprende a relacionar entradas con salidas. 3. Se evalúa con datos nuevos.
Usos principales	- Clasificación (asignar categorías). - Regresión (predecir valores numéricos).
Ejemplos	- Detectar spam en correos electrónicos. - Predecir el precio de una casa.- Reconocimiento de imágenes.
Requisitos	- Gran cantidad de datos etiquetados. - Etiquetas consistentes y precisas. - Datos representativos del problema.
Ventajas	- Buen rendimiento si los datos son adecuados. - Evaluación objetiva. - Ideal para problemas concretos con resultados esperados.
Desventajas	- Costoso en tiempo y recursos por la necesidad de etiquetado. - Poco flexible con datos no etiquetados. - Riesgo de sobreajuste si no se controla bien.





ENTRENAMIENTO SUPERVISADO: GIMNASIOS DISPONIBLES

Nombre	Plataforma	Tipo de entornos	Ventajas destacadas	URL oficial / Repositorio
Gymnasium (ex OpenAI Gym)	Python	Juegos simples, control clásico, Atari	Modular, ligero, comunidad activa, fácil de integrar con algoritmos estándar	https://gymnasium.farama.org
DeepMind Control Suite	Python + MuJoCo	Control motor, robótica	Física realista, útil para benchmarking	https://github.com/deepmind/dm_control
Unity ML-Agents	Unity + Python	Juegos 3D, simulaciones complejas	Gráficos 3D, multijugador, integración con Deep Learning	https://unity.com/products/machine-learning-agents
PyBullet	Python	Simulación robótica	Gratuito, física precisa, alternativa libre a MuJoCo	https://pybullet.org/ / https://github.com/bulletphysics/bullet3
MuJoCo	Python	Simulación avanzada de robots	Alta fidelidad, usado en investigación avanzada	https://mujoco.org
PettingZoo	Python	Multiagente (juegos y simulaciones)	Diseñado para múltiples agentes, integración con Gym y Stable-Baselines	https://www.pettingzoo.ml
Meta-World	Python + MuJoCo	Manipulación robótica multitarea	Reutilización de conocimiento entre tareas, entorno realista de manipulación	https://github.com/rlworkgroup/metaworld
MineRL	Python + Java	Minecraft	Ambiente naturalista, tareas jerárquicas, integración con aprendizaje profundo	https://minerl.io/ / https://github.com/minerllabs/minerl
BrainStorm Air Battle	Python + Java	Juego 2D Combate	Desarrollo de estrategias, combate, supervivencia.	

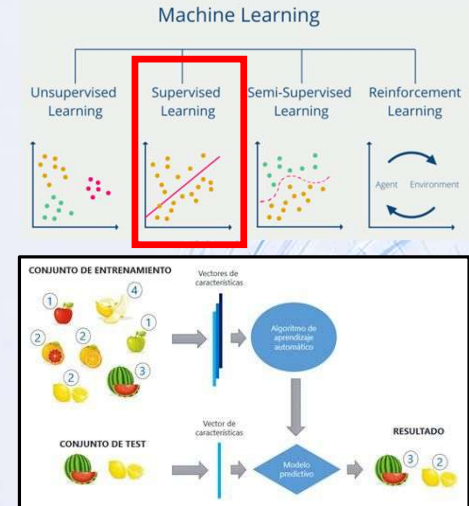




ENTRENAMIENTO SUPERVISADO [IMPLEMENTACION HIBRIDA]

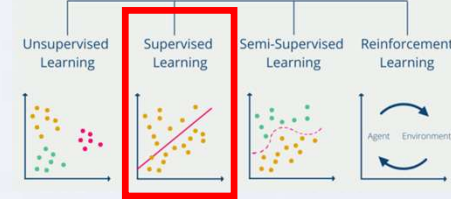
Etapas Inicial	Objetivo	Tareas específicas	Herramientas / Técnicas comunes	Errores frecuentes
1. Crear dataset manual	Generar un dataset inicial con ejemplos correctos de juego	Jugar partidas manualmente y registrar los estados (X) y comandos (Y)	CSV, pandas, grabación de partidas	Registrar casos incorrectos; Datos desbalanceados; pocas partidas para generalizar
2. Entrenar modelo inicial	Que la DNN aprenda a imitar el comportamiento humano	Definir arquitectura de la DNN, compilar, usar .fit() con dataset	TensorFlow/Keras, .fit(), pérdida binary_crossentropy	Overfitting por pocas partidas; aprender patrones erróneos si los datos son malos

Etapas Cíclicas	Objetivo	Tareas específicas	Herramientas / Técnicas comunes	Errores frecuentes
3. Ciclo de auto-juego y reentrenamiento	Refinar progresivamente la política del agente	Subdividido en pasos cíclicos		
3.1 Generar partidas automáticas	Que la DNN explore y produzca nuevas experiencias	Ejecutar partidas con el modelo actual; almacenar estados (X) y acciones (Y)	model.predict(), entorno AirBattle, listas o arrays para registrar episodios	Dataset demasiado repetitivo; falta de exploración de nuevas estrategias
3.2 Modificar dataset con reglas	Mejorar dataset aplicando heurísticas de desempeño	Revisar los estados y acciones; aplicar reglas programadas (ej: HEALTH<0.2 → CMD_BRAKE=1)	NumPy, Python, reglas if-else, reward shaping	Aplicar reglas incorrectas; sobre-corriger acciones; sesgo excesivo
3.3 Reentrenar modelo con dataset modificado	Ajustar la política de la DNN basándose en nuevas experiencias	Cargar modelo con load_model y entrenar con .fit() usando el dataset modificado	TensorFlow/Keras, .fit(), model.save/load	No normalizar entradas; entrenar con demasiadas pocas o demasiadas épocas; olvidar guardar modelo
3.4: Evaluación de métricas del modelo	Analizar qué tanto aprendió el modelo y cuáles entradas (inputs) tienen más impacto sobre las decisiones (outputs).	detectar variables inútiles, correlaciones falsas o comportamientos inesperados antes de lanzar otra ronda de autogeneración de partidas.	Análisis de impacto, Pesos Efectivos, Correlación Inputs outputs.	Confundir métricas de entrenamiento (loss, accuracy) con análisis de pesos, Interpretación equivocada de pesos, Omitir verificación con lógica del dominio.





Machine Learning



MANAGER ENTRENAMIENTO

El objetivo del script unificador es **orquestrar todas las etapas** para que el entrenamiento sea cíclico y autónomo:

- **Cargar el modelo inicial o crear uno nuevo.**
- **Generar partidas automáticas** (el modelo juega contra sí mismo).
- **Recolectar datasets** (ganador/perdedor, jugador 1/jugador 2).
- **Modificar y filtrar datasets** aplicando las reglas del dominio.
- **Reentrenar el modelo** con los datos actualizados.
- **Evaluar métricas** (impacto de entradas, correlaciones, pesos).
- **Guardar el modelo actualizado.**

Repetir el ciclo N veces o hasta que se cumplan criterios de mejora.

Funciones principales del MANAGER:

El script debería estructurarse en **módulos o funciones**, cada una llamando a los scripts ya existentes:

- `generar_partidas()` → llama al script que simula partidas y genera datasets.
- `modificar_datasets()` → procesa datasets y aplica reglas de consistencia.
- `entrenar_modelo()` → entrena o reentrena con el dataset combinado.
- `evaluar_metricas()` → ejecuta los scripts de análisis (impacto, correlación, pesos).
- `guardar_modelo()` → versiona el modelo (ej: `model_v2.h5`).

El **manager** las ejecuta en orden dentro de un bucle for ciclo in range(N):

Errores frecuentes en la unificación:

- **Rutas mal configuradas:** cada script puede estar guardando datasets en carpetas distintas; el manager debe unificar rutas.
- **Sobrescribir datasets o modelos:** si no se versionan correctamente, se pierden resultados previos.
- **No controlar la convergencia:** repetir ciclos sin analizar si realmente hay mejora puede llevar a sobreentrenamiento o estancamiento.
- **Falta de logs:** sin guardar un historial de métricas por ciclo, es imposible comparar la evolución.
- **Exceso de automatización:** no dejar puntos de control para revisión humana puede hacer que el modelo aprenda errores acumulados.

Rollback

X Y Z W 1200 H 1040

Start Training

1000.0 MS

01 X Y BOTW Trainer

1000.0 MS

01 X Y BOTW Evaluator

1000.0 MS

01 X Y BOTW Genetic Lab

1000.0 MS

GLAB TEST1

GLAB TEST2

GLAB TEST3

GLAB TEST4

Loop #

66

Boot Loading (s)

80

Total Time (hrs)

65

Time/Loop (H)

89

Total De Created

0

GLB DETEST Alive

0

GLB DETEST Created

0

GLB DETEST Destroyed

0

GLB DETEST Selected

10000001

GLB DETEST Created

0

GLB DETEST Alive

10000001

GLB DETEST MAX

0

GLB DETEST Threshold

0

Pause

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

1001

1002

1003

1004

1005

1006

1007

1008

1009

1010

1011

1012

1013

1014

1015

1016

1017

1018

1019

1020

1021

1022

1023

1024

1025

1026

1027

1028

1029

1030

1031

1032

1033

1034

1035

1036

1037

1038

1039

1040

1041

1042

1043

1044

1045

1046

1047

1048

1049

1050

1051

1052

1053

1054

1055

1056

1057

1058

1059

1060

1061

1062

1063

1064

1065

1066

1067

1068

1069

1070

1071

1072

1073

1074

1075

1076

1077

1078

1079

1080

1081

1082

1083

1084

1085

1086

1087

1088

1089

1090

1091

1092

1093

1094

1095

1096

1097

1098

1099

1100

1101

1102

1103

1104

1105

1106

1107

1108

1109

1110

1111

1112

1113

1114

1115

1116

1117

1118

1119

1120

1121

1122

1123

1124

1125

1126

1127

1128

1129

1130

1131

1132

1133

1134

1135

1136

1137

1138

1139

1140

1141

1142

1143

1144

1145

1146

1147

1148

1149

1150

1151

1152

1153

1154

1155

1156

1157

1158

1159

1160

1161

1162

1163

1164

1165

1166

1167

1168

1169

1170

1171

1172

1173

1174

1175

1176

1177

1178

1179

1180

1181

1182

1183

1184

1185

1186

1187

1188

1189

1190

1191

1192

1193

1194

1195

1196

1197

1198

1199

1200

1201

1202

1203

1204

1205

1206

1207

1208

1209

1210

1211

1212

1213

1214

1215

1216

1217

1218

1219

1220

1221

1222

1223

1224

1225

1226

1227

1228

1229

1230

1231

1232

1233

1234

1235

1236

1237

1238

1239

1240

1241

1242

1243

1244

1245

1246

1247

1248

1249

1250

1251

1252

1253

1254

1255

1256

1257

1258

1259

1260

1261

1262

1263

1264

1265

1266

1267

1268

1269

1270

1271

1272

1273

1274

1275

1276

1277

1278

1279

1280

1281

1282

1283

1284

1285

1286

1287

1288

1289

1290

1291

1292

1293

1294

1295

1296

1297

1298

1299

1300

1301

1302

1303

1304

1305

1306

1307

1308

1309

1310

1311

1312

1313

1314

1315

1316

1317

1318

1319

1320

1321

1322

1323

1324

1325

1326

1327

1328

1329

1330

1331

1332

1333

1334

1335

1336

1337

1338

1339

1340

1341

1342

1343

1344

1345

1346

1347

1348

1349

1350

1351

1352

1353

1354

1355

1356

1357

1358

1359

1360

1361

1362

1363

1364

1365

1366

1367

1368

1369

1370

1371

1372

1373

1374

1375

1376

1377

1378

1379

1380

1381

1382

1383

1384

1385

1386

1387

1388

1389

1390

1391

1392

1393

1394

1395

1396

1397

1398

1399

1400

1401

1402

1403

1404

1405

1406

1407

1408

1409

1410

1411

1412

1413

1414

1415

1416

1417

1418

1419

1420

1421

1422

1423

1424

1425

1426

1427

1428

1429

1430

1431

1432

1433

1434

1435

1436

1437

1438

1439

1440

1441

1442

1443

1444

1445

1446

1447

1448

1449

1450

1451

1452

1453

1454

1455

1456

1457

1458

1459

1460

1461

1462

1463

1464

1465

1466

1467

1468

1469

1470

1471

1472

1473

1474

1475

1476

1477

1478

1479

1480

1481

1482

1483

1484

1485

1486

1487

1488

1489

1490

1491

1492

1493

1494

1495

1496

1497

1498

1499

1500

1501

1502

150



Etapa 1: Crear Dataset Manual

Generar un dataset inicial que contenga ejemplos correctos de juego, es decir, pares **estado** → **acción** donde la acción tomada sea razonablemente buena. Este dataset servirá como base para entrenar la red neuronal antes de que aprenda a jugar por sí misma.

Tareas específicas:

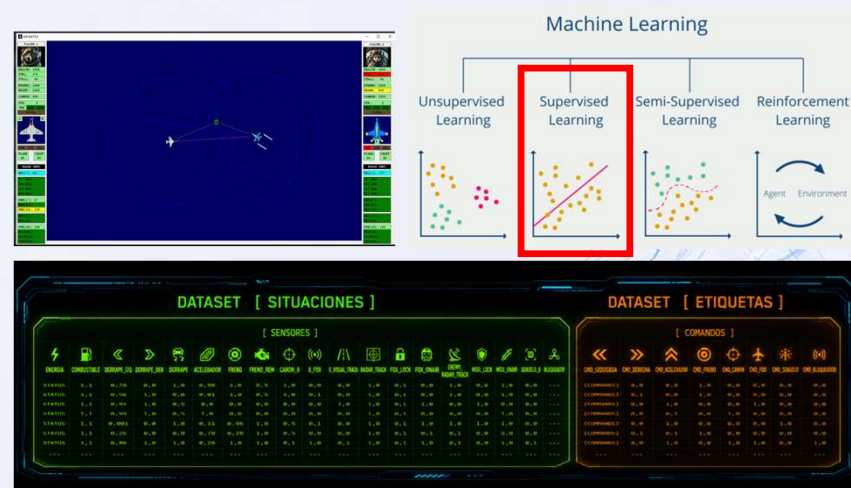
- Jugar partidas manualmente en AirBattle.
- Asegurarse de que los datos cubran distintos escenarios: atacar, esquivar, recolectar ítems, etc.
- Crear DataSet unificado.

Herramientas / Técnicas comunes:

- CSV o archivos Excel para almacenar datos.
- Pandas para manipular y limpiar los datasets.
- Scripts en Python para convertir logs del juego en tablas de estado y comandos.

Errores frecuentes:

- **Pocas partidas para generalizar:** El dataset solo refleja tus decisiones en un tipo de escenario, la red no aprende situaciones diversas.
- **Sesgo del jugador:** Si siempre juegas agresivo o pasivo, la DNN puede aprender una estrategia limitada.
- **Duplicación de datos:** Filas repetidas que sobre-representan ciertos estados y distorsionan el aprendizaje.
- **Falta de exploración de casos extremos:** No incluir situaciones críticas (salud baja, enemigos cercanos, pocos recursos) que son importantes para que la DNN aprenda a reaccionar.
- **Falta de diversidad de escenarios:** Si todas las partidas ganadas se obtienen con la misma estrategia, la DNN no aprenderá a adaptarse a situaciones distintas.



```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```



Etapla 2: Entrenar modelo inicial

Que la DNN aprenda a **imitar las decisiones correctas de las partidas manuales**, utilizando los datasets generados por AirBattle (ganador y perdedor). Esta etapa establece la **política inicial** del agente antes de que comience a jugar contra sí mismo y a refinarse.

Tareas específicas:

Definir la arquitectura de la DNN:

Entradas: 40 sensores del avión.

Capas ocultas: p.ej., 2 capas densas con 10 neuronas cada una.

Salidas: 8 comandos (CMD_LEFT, CMD_RIGHT, ...).

Preparar los datos:

Leer los CSV de ganador y perdedor.

Normalizar las entradas (HEALTH, FUEL, etc.) para que los valores estén en rangos compatibles con la función de activación (sigmoid).

Opcional: ponderar los ejemplos de ganadores más alto que los perdedores para reforzar buenas acciones.

Compilar el modelo:

Seleccionar **optimizador** (adam, sgd).

Seleccionar **función de pérdida** adecuada (binary_crossentropy) porque las salidas son acciones binarias.

Entrenar con .fit():

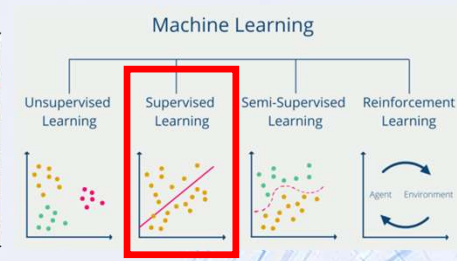
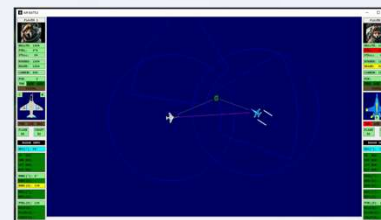
Ajustar el modelo usando el dataset manual.

Definir número de épocas, tamaño de batch y validación si es necesario.

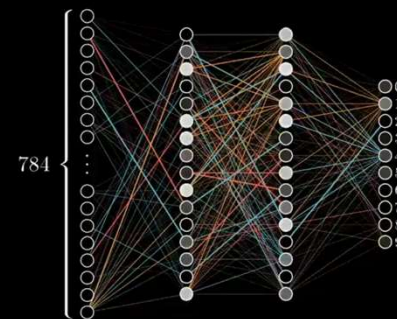
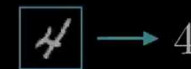
Guardar el modelo entrenado:

Para poder cargarlo en etapas posteriores y continuar el entrenamiento iterativo.

```
model.save("airbattle_model_initial.keras")
```



Training in progress. . .



```
SCRIPT >>> "TRAIN03_AIRBATTLE_ENTRENAMIENTO.py"
```



Etapa 2: Entrenar modelo inicial

Que la DNN aprenda a **imitar las decisiones correctas de las partidas manuales**, utilizando los datasets generados por AirBattle (ganador y perdedor). Esta etapa establece la **política inicial** del agente antes de que comience a jugar contra sí mismo y a refinarse.

Errores frecuentes

Overfitting por pocas partidas:

La DNN memoriza patrones específicos de las partidas manuales y no generaliza bien a nuevas situaciones.

Dataset desequilibrado:

Si hay muchos más ejemplos de perdedor que de ganador, la DNN puede aprender estrategias incorrectas.

No normalizar las entradas:

Valores como HEALTH o FUEL en escalas muy diferentes pueden hacer que la DNN aprenda lentamente o mal.

Arquitectura inadecuada:

Muy pocas neuronas/capas → incapaz de aprender patrones complejos.

Muy grande → entrenamiento lento y riesgo de overfitting.

Funciones de activación incorrectas:

Usar relu en la capa de salida para acciones binarias puede producir resultados no binarios; se recomienda sigmoid o softmax según el enfoque.

Batch o número de épocas inapropiado:

Demasiadas épocas → sobreajuste; pocas épocas → aprendizaje incompleto.

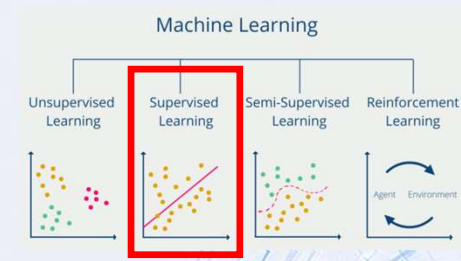
Tamaño de batch demasiado grande → poca variabilidad por actualización; demasiado pequeño → entrenamiento ruidoso.

No validar durante entrenamiento:

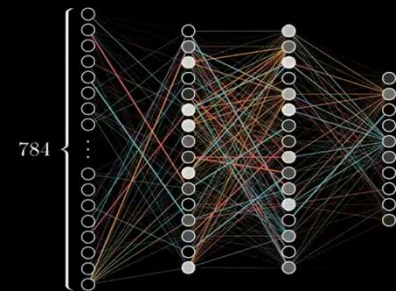
Sin un dataset de validación, no se detecta si el modelo está aprendiendo correctamente o simplemente memorizando ejemplos.

Ignorar secuencia temporal:

.fit() trata cada fila como independiente; si el juego tiene secuencia de estados, la DNN podría no capturar ciertas dinámicas temporales (esto se corrige más adelante con reglas o RL).



Training in progress...



```
SCRIPT >>> "TRAIN03_AIRBATTLE_ENTRENAMIENTO.py"
```




Etapla 3.1: Generar partidas automáticas

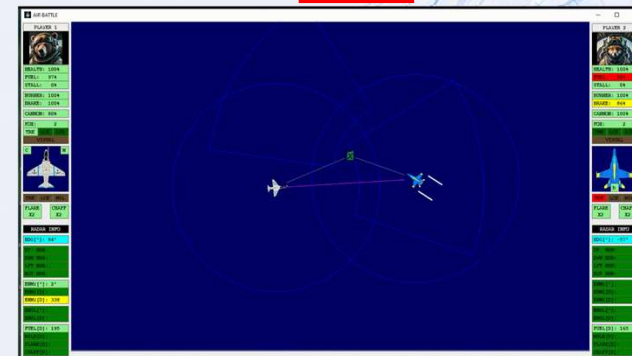
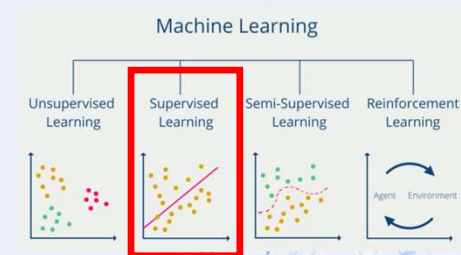
Permitir que la DNN juegue contra sí misma para **producir nuevas experiencias** que luego se usarán para mejorar la política del agente. Esto permite que la red aprenda a enfrentarse a situaciones más diversas que las observadas en las partidas manuales.

Tareas específicas:

- **Cargar el modelo entrenado inicialmente:** Se usa el modelo entrenado en la Etapa 2.
- **Configurar el entorno AirBattle:**
 - Inicializar la partida para que ambos jugadores sean controlados por la DNN.
 - Registrar cada frame los estados (X) y las acciones tomadas (Y).
- **Ejecutar la partida automáticamente:**
 - En cada paso del juego, predecir las acciones usando `model.predict(state)`.
 - Aplicar un umbral (p.ej., >0.5) para decidir si activar cada comando.
 - Guardar estados y comandos en arrays o listas.
- **Al finalizar la partida:** AirBattle genera dos datasets automáticamente: uno para el **ganador** y otro para el **perdedor**.

Errores frecuentes

- **Dataset demasiado repetitivo:** Si la DNN juega siempre igual, los estados y acciones generados no aportan información nueva.
- **Falta de exploración:** La DNN puede quedarse atrapada en estrategias subóptimas ("política local"), porque solo replica lo aprendido.
- **Desalineación de estados y acciones:** Guardar un estado con la acción de un frame diferente puede corromper el dataset.
- **Errores de sincronización con el juego:** Latencia o retrasos en la predicción pueden generar acciones aplicadas tarde o inconsistentes.
- **Sobrepeso de un jugador:** Si uno de los jugadores DNN es consistentemente más fuerte, el dataset del perdedor podría ser demasiado amplio y no reflejar situaciones útiles.
- **Fallos en la predicción de la DNN:** Salidas continuas sin binarizar correctamente los comandos pueden generar datos incorrectos.
- **Archivos corruptos o no guardados:** Partida interrumpida por error del juego o fallo de disco puede provocar datasets incompletos.
- **Ignorar diversidad de escenarios:** Si siempre se inicializa la partida de la misma manera, los datos generados no cubren todas las posibles situaciones del juego.



```
SCRIPT >>> "TRAIN01_EJECUTADOR_AIR_BATTLE.py"
```

```
SCRIPT >>> "COMMS01_KB_PLAYER.py"
```

```
SCRIPT >>> "COMMS02_AI_PLAYER.py"
```



Etapla 3.2: OPTIMIZACION del DATASET

Objetivo:

Refinar el dataset generado por las partidas automáticas para **corregir errores, eliminar datos irrelevantes y aplicar reglas heurísticas** que mejoren la política de la DNN antes del reentrenamiento.

Tareas específicas:

Unificar datasets de partidas ganadoras:

- Recorrer la carpeta de archivos .Battle generados por AirBattle.
- Concatenar todos los archivos en un único dataset unificado (Dataset_Unificado.txt).

Depuración de datos:

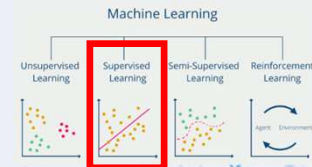
- Leer el dataset unificado y **balancear** las diferentes situaciones para que cada una tenga un **proporción** representativa.
- Contar estadísticas de filas totales, descartadas y conservadas.

Aplicar reglas heurísticas sobre los datos:

- **Ejemplo 1:** Si $STALL > 0.7$, desactivar comandos de giro (CMD_LEFT y CMD_RIGHT) para evitar perder control del avión.
- **Ejemplo 2:** (ya incluido) filas con todos los comandos en cero se descartan automáticamente. Estas reglas **mejoran la calidad del dataset**, asegurando que la DNN no aprenda comportamientos peligrosos o inútiles.

Errores frecuentes en esta etapa

- **Reglas demasiado estrictas:** Aplicar correcciones excesivas puede eliminar variabilidad útil y limitar la capacidad de la DNN de aprender estrategias alternativas.
- **Descartar filas útiles:** Filas donde todos los comandos son cero podrían representar situaciones de espera estratégica; descartarlas siempre puede eliminar casos válidos.



DATASET [SITUACIONES]														
[SENSORES]														
WING	WING	WING	WING	WING	WING	WING	WING	WING	WING	WING	WING	WING	WING	WING
STATUS	1.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
STATUS	1.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
STATUS	1.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
STATUS	1.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
STATUS	1.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
STATUS	1.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
STATUS	1.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
STATUS	1.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
STATUS	1.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1

DATASET [ETIQUETAS]														
[COMANDOS]														
CMD_LEFT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
CMD_RIGHT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
CMD_FORWARD	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
CMD_BACKWARD	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
CMD_UP	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
CMD_DOWN	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
CMD_TURN_LEFT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
CMD_TURN_RIGHT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
CMD_FIRE	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
CMD_REPAIR	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Un dataset de partidas crudas nunca está listo para entrenar una DNN.
Contiene:

- Ruido humano
- Filas sin información
- Atajos espurios
- Desequilibrios entre situaciones frecuentes y raras.

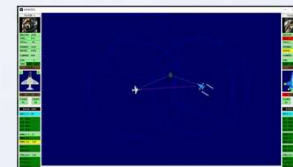
La Etapa 3 —**Dataset Engineering**— es donde se transforma ese material crudo en un dataset apto para *Behavior Cloning*, aplicando un conjunto organizado de técnicas que la comunidad de Imitation Learning usa de forma estándar.

Antes de entrar en las técnicas, una distinción conceptual que atraviesa toda la etapa:

- **Regla:** Una condición puntual y aislada (si X entonces Y).
- **Política:** Un conjunto organizado de reglas que persigue un objetivo coherente (seguridad, navegación, combate, recuperación...).
- **SIT (Situación):** ¿La condición física ocurrió? (ej.: hay una amenaza dentro de rango)
- **COR (Corrección):** Dado que ocurrió, ¿hizo falta además modificar el comando para resolverla?

Con esto en mente, la Etapa 3 se organiza en las siguientes presentaciones:

- Ingestion y Dataset Cleaning
- Policy Engineering (inyección de conocimiento experto)
- Anti-Leakage Sanitization
- Dataset Filtering y Dataset Balancing
- Label Engineering
- Action Masking
- Dataset Randomization y Dataset Validation

[illegible]

```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```




Etapas 3.2: OPTIMIZACION del DATASET

Ingestion and Normalization

Objetivo

Convertir múltiples partidas registradas por el simulador en un único dataset consistente, y dejar los datos en un estado "neutro" antes de aplicar ningún criterio experto.

Tareas específicas

Unificación de partidas:

Recorrer la carpeta de partidas ganadoras generadas por el simulador.
Concatenar todas en un único dataset maestro, respetando un esquema de columnas fijo (no el de cada archivo individual, que puede variar levemente entre versiones del simulador).

Normalización / Feature Scaling:

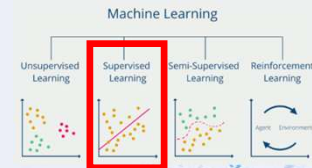
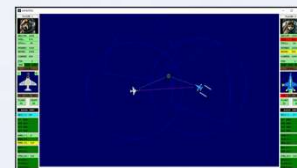
Cada sensor físico tiene su propia escala natural (energía, combustible, ángulos, distancias). Antes de que una DNN pueda aprender de ellos hace falta llevarlos a un rango común.

Los sensores angulares (que oscilan entre un valor negativo y uno positivo simétrico) no se escalan igual que los sensores de magnitud pura (que van de 0 a un máximo): **tratarlos con la misma fórmula genérica introduce sesgos.**

Errores frecuentes en esta etapa:

Confundir Cleaning con Filtering: cleaning prepara la fila, no decide si se queda o se va.

Aplicar la misma fórmula de normalización a sensores angulares y de magnitud: produce compresiones o expansiones de rango incorrectas.



DATASET [SITUACIONES]															
[SENSORES]															
ENERGIA	COMBUSTIBLE	ANGULO DE EMPUJOS	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE
STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS
ENERGIA	COMBUSTIBLE	ANGULO DE EMPUJOS	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE
STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS
ENERGIA	COMBUSTIBLE	ANGULO DE EMPUJOS	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE
STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS
ENERGIA	COMBUSTIBLE	ANGULO DE EMPUJOS	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE
STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS
ENERGIA	COMBUSTIBLE	ANGULO DE EMPUJOS	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE	ANGULO DE GIRE
STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS	STATUS

SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"



Etapas 3.2: OPTIMIZACION del DATASET

Policy Engineering (Inyección de Conocimiento Experto)

Objetivo

Reemplazar el comando crudo del humano por una etiqueta generada por reglas expertas organizadas en políticas, de forma que la DNN aprenda una política *consistente* y no los errores o inconsistencias de un jugador humano cansado, distraído o simplemente subóptimo.

Concepto clave: jerarquía de políticas

En cualquier instante, más de una política podría "querer" tomar el control de la dirección del avión (evadir una amenaza, perseguir un objetivo, buscar un recurso...). Como el avión solo puede girar en una dirección a la vez, hace falta un

mecanismo de arbitraje: una política de prioridad más alta gana y bloquea a las de menor prioridad para ese ciclo. Diseñar bien este orden de prioridades es, en sí mismo, una decisión de diseño tan importante como las reglas individuales.

Tareas específicas

Definir, para cada familia, la condición de

- **SIT** (cuándo se considera que la situación está presente)
- **COR** (cuándo, dada la situación, hace falta modificar el comando).

Implementar el arbitraje de prioridad de forma explícita (una bandera o mecanismo equivalente que impida que una política de menor prioridad sobrescriba a una de mayor prioridad ya aplicada en ese ciclo).

Documentar el orden de prioridades elegido y su justificación táctica.

Errores frecuentes en esta etapa:

Reglas demasiado estrictas: sobre-corriger eliminar variabilidad útil y le quita a la DNN la posibilidad de aprender estrategias alternativas válidas.

- **Ausencia de arbitraje de prioridades:** si dos políticas escriben el mismo comando sin un orden definido, la última que se ejecuta "gana" por accidente, no por diseño.
- **No medir SIT/COR por separado:** sin esta distinción es imposible saber después si una política está sobre-representada porque la situación es común, o porque se está corrigiendo de más.

Familia	Qué resuelve	Prioridad típica
Políticas de Seguridad	Amenazas que comprometen la supervivencia inmediata (proyectiles entrantes, límites del área de juego)	Máxima
Políticas de Recuperación	Estados físicos degradados del propio agente (pérdida de control, sobrecalentamiento, etc.) que deben corregirse sin importar qué más esté pasando	Muy alta
Políticas Tácticas	Uso de armamento u ofensiva (disparo, bloqueo de blancos)	Media-alta
Políticas Defensivas	Contramedidas ante una amenaza detectada (señuelos, interferencia)	Media-alta
Políticas de Navegación	Persecución u orientación hacia un objetivo (enemigo, punto de interés)	Media
Políticas de Gestión de Recursos	Recolección de ítems o recursos del entorno	Baja

```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```



Etapa 3.2: OPTIMIZACION del DATASET

Anti-Leakage Sanitization

Objetivo: Detectar y neutralizar sensores que funcionan como un "espejo" del comando que se quiere predecir, para que la red aprenda la relación real estado → acción y no un atajo trivial.

Concepto clave: Causal Confusion

- Es un proceso de higiene de datos que se aplica *después* de que las políticas ya resolvieron los comandos.
- En la literatura de Imitation Learning se lo conoce como mitigar la **Causal Confusion** (De Haan et al., 2019): cuando un sensor está prácticamente correlacionado 1 a 1 con el comando que se quiere predecir, la red aprende a copiarlo en vez de aprender la causa real de la acción.
- Esa "muleta" desaparece —o directamente falla— apenas el modelo tiene que volar solo, sin ese comando previo disponible como atajo.
- Es una de las causas más comunes de que un modelo funcione perfecto en entrenamiento y colapse en inferencia.

Tipos de fuga a buscar

- **Sensores de retorno directo:** variables que en la práctica replican el comando del paso anterior. Aportan cero información causal nueva y deberían neutralizarse sistemáticamente.
- **Sensores acoplados por construcción física:** variables que crecen o decrecen *mientras* se mantiene presionado cierto comando, generando una correlación espuria fuerte aunque no sean un espejo exacto. Aquí la solución no siempre es "apagar siempre": conviene preguntarse si, por debajo de cierto umbral de relevancia, el sensor es solo ruido correlacionado (se neutraliza), pero por encima de ese umbral pasa a ser una señal de seguridad real (se preserva, e incluso puede justificar forzar un override de seguridad sobre el comando).

Tareas específicas

- Auditar el esquema de sensores en busca de candidatos a fuga (buscar correlaciones altas entre sensor y comando correspondiente).
- Clasificar cada candidato: ¿es una fuga pura (se neutraliza siempre) o una fuga condicional (se neutraliza solo por debajo de un umbral de relevancia física)?
- Registrar estadísticas de cuántas veces se aplicó cada neutralización/override, para poder auditarlas.

Errores frecuentes en esta etapa

- **No distinguir fuga pura de fuga condicional:** apagar un sensor que en niveles extremos es información de seguridad real elimina señal valiosa, no solo ruido.
- **Aplicar la sanitización antes de que las políticas terminen de resolver los comandos:** el orden importa, porque la sanitización necesita el comando ya definitivo para decidir si corresponde un override.
- **Ignorar esta etapa por completo:** es la causa número uno de que un modelo entrenado con altísima métrica offline se comporte de forma errática al volar de forma autónoma.

```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```

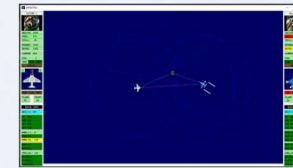

Dataset Filtering y Dataset Balancing

Asegurar que el dataset final contenga solo filas con información de aprendizaje real, y que ninguna política domine el entrenamiento simplemente por ser más frecuente que las demás.

- **Qué se descarta:** filas donde ninguna política terminó activando ningún comando (el agente "no hizo nada" desde el punto de vista de la política). Estas filas no le enseñan nada nuevo a la red sobre la relación estado → acción.

- **Qué se reporta:** cantidad de filas totales, descartadas y conservadas, para poder auditar qué porcentaje del dataset original terminó siendo útil.

- **Descartar filas útiles:** una fila con todos los comandos en cero podría representar una situación legítima de "espera estratégica" (por ejemplo, mantener rumbo porque nada amerita corrección). Descartarlas siempre, sin matices, puede eliminar casos válidos que la red debería poder reproducir.
- **Filtrar antes de que las políticas terminen de escribir los comandos:** si se filtra sobre el dataset crudo en vez del dataset con políticas ya aplicadas, se descartan filas por el criterio equivocado.

[illegible]

```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```

Dataset Filtering y Dataset Balancing

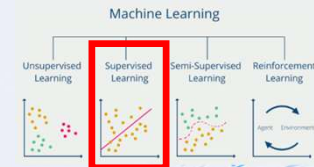
Las políticas no ocurren con la misma frecuencia: navegación y persecución son comunes; evasión de amenazas o recolección de ítems raros son escasas. Sin balanceo, la red tiende a converger hacia la política dominante y a ignorar las políticas minoritarias, que suelen ser justamente las más críticas (seguridad).

- **Oversampling:** duplicar o repetir (con o sin variaciones) las filas de las políticas minoritarias.

- **Undersampling:** reducir la cantidad de filas de la política dominante para emparejar la proporción.
- **Reweighting:** en vez de tocar el dataset, ajustar el peso de cada fila en la función de pérdida según la frecuencia de su política.
- **Balance temporal:** evitar que partidas muy largas (con muchas filas de navegación repetitiva) pesen desproporcionadamente frente a partidas cortas pero ricas en situaciones variadas.

- **Balancear antes de filtrar:** si se balancea sobre datos que todavía incluyen filas vacías, se termina sobre-representando ruido.

- **Oversampling ingenuo sin variación:** repetir filas idénticas sin ningún tipo de perturbación puede llevar a que la red memorice en vez de generalizar.
- **Balancear por política ignorando la superposición:** una misma fila puede pertenecer a más de una política simultáneamente (por el mecanismo de prioridades); balancear ingenuamente por conteo de "comando activo" puede distorsionar la proporción real de situaciones.



The screenshot shows the 'Dataset' tab with the 'Situaciones' (Situations) sub-tab selected. The interface includes a top navigation bar with 'Dataset' and 'Situaciones' tabs. Below this is a 'Sensores' (Sensors) section with icons for various sensors: FUELO, GASOLINA, TEMPERATURA, HUMEDAD, VIBRACION, PRESION, VELOCIDAD, ALTURA, DISTANCIA, TIEMPO, and others. The main area displays a table with columns for each sensor and rows for different situations. The table is organized into two main sections: 'Situaciones' and 'Etiquetas' (Labels). The 'Situaciones' section lists 10 situations (SITUACION 1 to SITUACION 10) with corresponding values for each sensor. The 'Etiquetas' section lists 10 labels (ETIQUETA 1 to ETIQUETA 10) with corresponding values for each sensor. The table is organized into two main sections: 'Situaciones' and 'Etiquetas' (Labels).

```
SCRIPT >>> "TRAIN02 UNIFICADOR DATASET.py"
```

```
SCRIPT >>> "TRAIN02 UNIFICADOR DATASET.py"
```



```
SCRIPT >>> "TRAIN02 UNIFICADOR DATASET.py"
```


ion y Dataset Validation

garando que el orden de las filas no introduzca sesgos de aprendizaje, y que existan
ra decidir si el dataset está listo para entrenamiento.

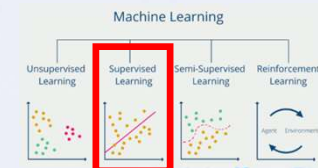
ization

- por naturaleza, ordenadas cronológicamente. Si ese orden se preserva, la red puede capturar correlaciones temporales espurias (qué viene "después de qué") en vez de la causalidad $X \rightarrow Y$. Para evitar esto, se mezclan (shuffle) las filas una vez que todas las etapas anteriores ya se aplicaron, de modo que cada fila se evalúe de forma independiente durante el entrenamiento.

2

Además del pipeline, conviene reportar un conjunto de métricas de salud del dataset:

- conservadas vs. descartadas en cada etapa.
 ración de cada política (idealmente expresada como SIT y COR por separado, como se
 tación 2).
 des aplicados por Anti-Leakage Sanitization.
 líticas de menor prioridad: ¿aparecen con suficiente frecuencia como para que la red
 representativos, o son tan raras que probablemente necesiten Balancing?
 nciona como un "gate" de calidad: si alguna política crítica de seguridad tiene muy
 una señal de que hace falta volver a alguna etapa anterior (más partidas, más
 de umbrales) antes de pasar a entrenamiento.



The image shows a futuristic control panel with two main sections: **DATASET [SITUACIONES]** and **DATASET [ETIQUETAS]**.

DATASET [SITUACIONES] features a row of 16 sensor icons labeled **[SENSORES]**. Below this is a table with 16 columns and 6 rows of data. The columns are: TEMP, HUMID, PRESS, ACIDITY, ALKALINITY, PHOS, PHOS_PO4, CATAL_3, E_FO2, E_FO2_100, TEMP_TRAN, PHOS_PO4_TRAN, PHOS_PO4_TRAN_100, PHOS_PO4_TRAN_100_100, PHOS_PO4_TRAN_100_100_100, PHOS_PO4_TRAN_100_100_100_100, PHOS_PO4_TRAN_100_100_100_100_100, and PHOS_PO4_TRAN_100_100_100_100_100_100.

DATASET [ETIQUETAS] features a row of 10 command icons labeled **[COMANDOS]**. Below this is a table with 10 columns and 6 rows of data. The columns are: DIR_000000, DIR_00001, DIR_00002, DIR_00003, DIR_00004, DIR_00005, DIR_00006, DIR_00007, DIR_00008, and DIR_00009.

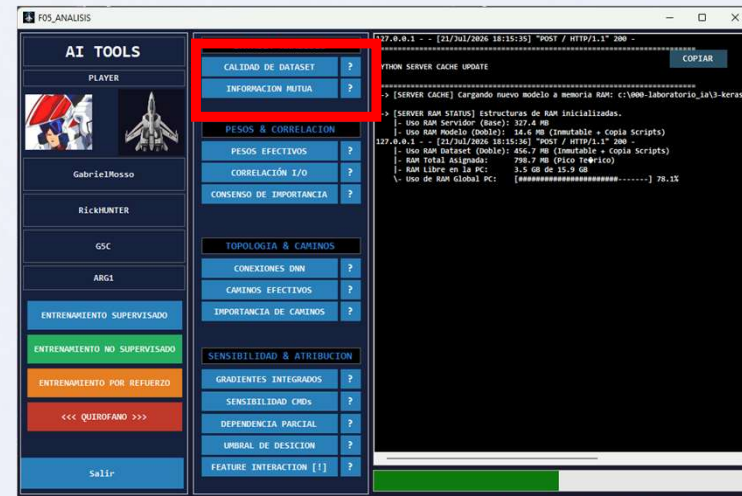
```
SCRIPT >>> "TRAIN02 UNIFICADOR DATASET.py"
```



Etapas 3.3: Análisis del DATASET.

Script: CHECK_DA_01_CALIDAD_DEL_DATASET.py

Objetivo general: auditar la calidad del dataset de entrenamiento desde múltiples ángulos: balance de clases, ruido en las decisiones, estabilidad temporal y complejidad de las etiquetas.



SCRIPT >>> "CHECK_DA_01_CALIDAD_DEL_DATASET.py"



Etap 3.3: Análisis del DATASET.

Figura 1 — Balance de Comandos + Co-ocurrencia

Subplot izquierdo: Balance General de Comandos (barh)

Muestra la **frecuencia de uso (%)** de cada comando (CMD_*) sobre el total de filas del dataset.

Cálculo: $\text{df}[\text{columnas_comando}].\text{mean}() * 100$.

Para qué se usa: detectar **desbalance de clases**.

Un comando usado <1% de las veces = casi sin señal positiva para entrenar.

Riesgo directo: recall bajo en esa clase específica.

Subplot derecho: Heatmap de Co-ocurrencia

Matriz de **probabilidad condicional**: $P(\text{columna}=1 \mid \text{fila}=1)$.

Cálculo: $(\text{activaciones simultáneas}) / (\text{activaciones de la fila})$.

No es simétrica — se lee direccionalmente: "cuando aprieto FILA, ¿qué % de las veces también aprieto COLUMNA?".

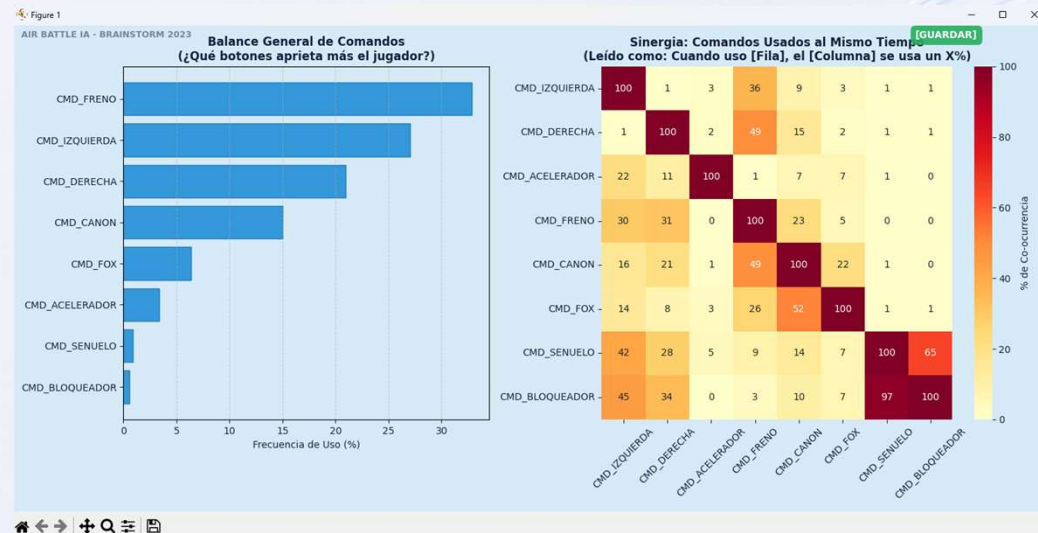
Para qué se usa: detectar **sinergias reales de gameplay** (ej. FRENO + DERECHA en curvas).

Importa porque en multilabel el modelo debe aprender combinaciones, no solo clases aisladas.

Relación con otras figuras:

Es la base de la **Figura 5** (detalla combinaciones exactas).

Se resume numéricamente en la **Figura 4** (cardinalidad, sin decir cuáles se combinan).





Etapla 3.3: Análisis del DATASET.

Figura 2 — Target Conflict (Ruido en las Decisiones)

Usa **KDTree (KNN)** sobre sensores normalizados (z-score) para encontrar el vecino más cercano de cada fila en el espacio de estado.

Si distancia $< 0.5 \rightarrow$ se considera "misma situación".

Compara si el jugador tomó **la misma decisión** en ambos casos.

Subplot izquierdo: Torta (Consistencia del Jugador)

% de pares "casi idénticos" con decisiones **consistentes** vs **contradictorias**.

Subplot derecho: Panel explicativo

Define el criterio: **>15% de conflicto = ruido significativo**.

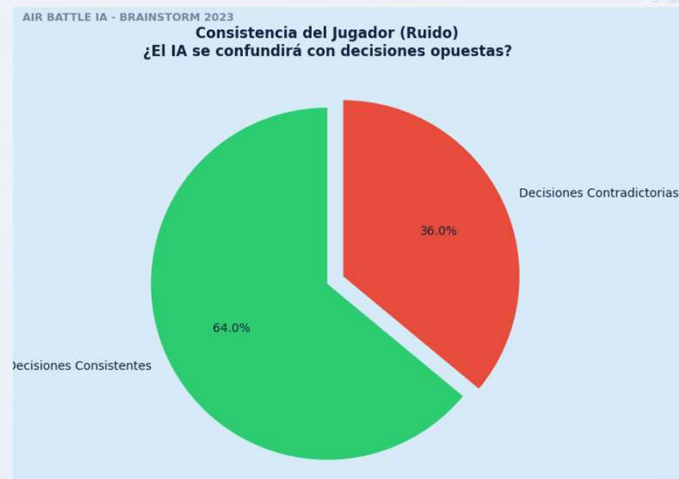
Para qué se usa: medir si el dataset enseña **dos respuestas correctas para la misma pregunta**.

Esto pone un **techo duro** a la accuracy alcanzable, sin importar qué tan bueno sea el modelo.

Relación con otras figuras:

Distinto de la Figura 1: no mide balance, mide **contradicción etiqueta-vs-estado**.

⚠ **Nota técnica:** el umbral 0.5 es fijo y arbitrario respecto a 40 sensores normalizados — conviene calibrarlo si el % de conflicto sale sospechosamente alto o bajo.





Eta3a 3.3: Análisis del DATASET.

Figura 3 — Estabilidad Temporal (Data Drift)

(Proceso 5)

Divide el dataset (ya en orden cronológico) en **50 bloques secuenciales**.

Grafica frecuencia (%) de cada CMD_* y SIT_* bloque a bloque.

Líneas **sólidas** = comandos | Líneas **punteadas** = situaciones.

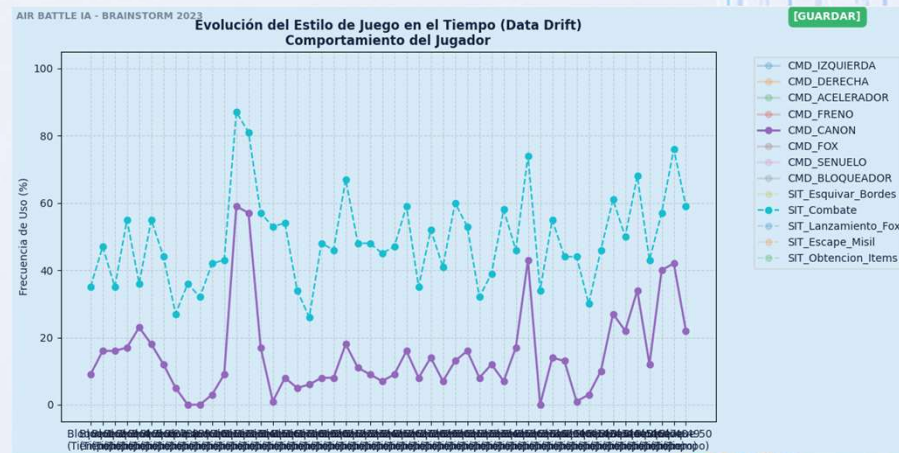
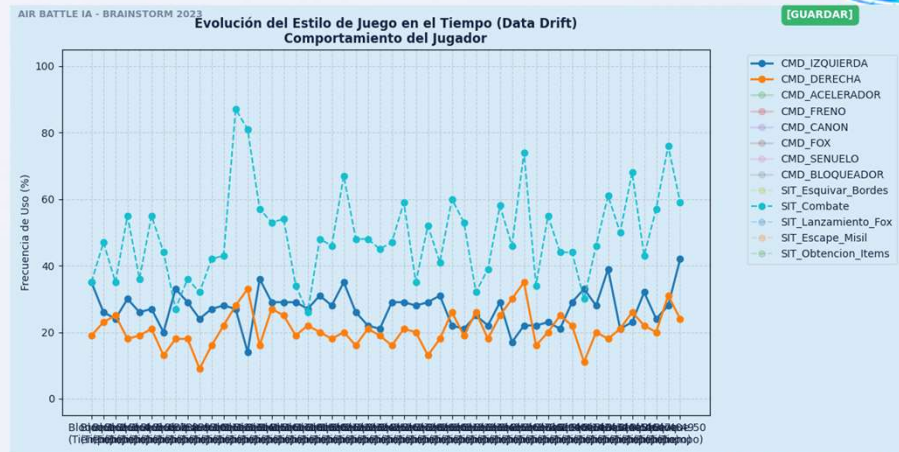
Interactivo: click en la leyenda oculta/muestra series (por defecto solo visibles: CMD_IZQUIERDA, CMD_DERECHA y las SIT_*).

Para qué se usa: detectar si el **estilo de juego cambió con el tiempo**.

Ej: jugador mejoró, cambió de estrategia, o se mezclaron sesiones distintas.

Relación con otras figuras:

Complementa temporalmente a la **Figura 1**: la Fig. 1 da el promedio global, esta muestra si ese promedio es estable o un artefacto de mezclar comportamientos distintos.





Etapla 3.3: Análisis del DATASET.

Figura 4 — Complejidad de Decisiones (Cardinalidad)

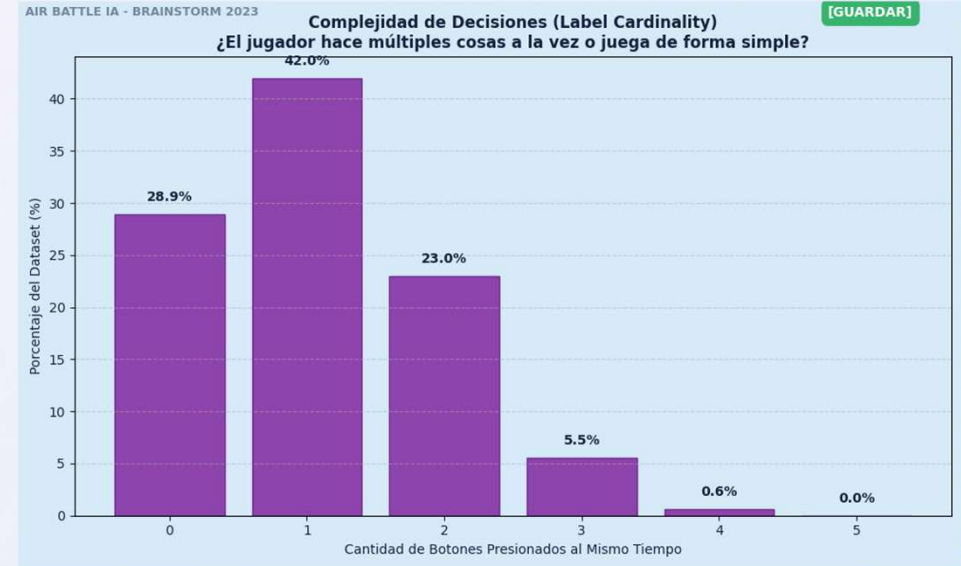
Histograma de `df[columnas_comando].sum(axis=1)`: cuántos botones se presionan **simultáneamente** por fila (0, 1, 2, 3...).

Métrica estándar de "label cardinality" en literatura multilabel.

Para qué se usa: dimensionar qué tan compleja es la salida que el modelo debe predecir por muestra.

Relación con otras figuras:

Resume en un solo número lo que la **Figura 1** (heatmap) muestra en detalle par a par. Precede a la **Figura 5**, que detalla combinaciones exactas.





Etap 3.3: Análisis del DATASET.

Figura 5 — Combinaciones Activas (Powerset Imbalance)

Concatena comandos activos por fila en un string (ej. "IZQUIERDA + FOX").

Excluye filas sin ningún comando activo ("NINGUNO").

Calcula frecuencia de cada combinación única sobre el subconjunto de filas activas.

Grafica:

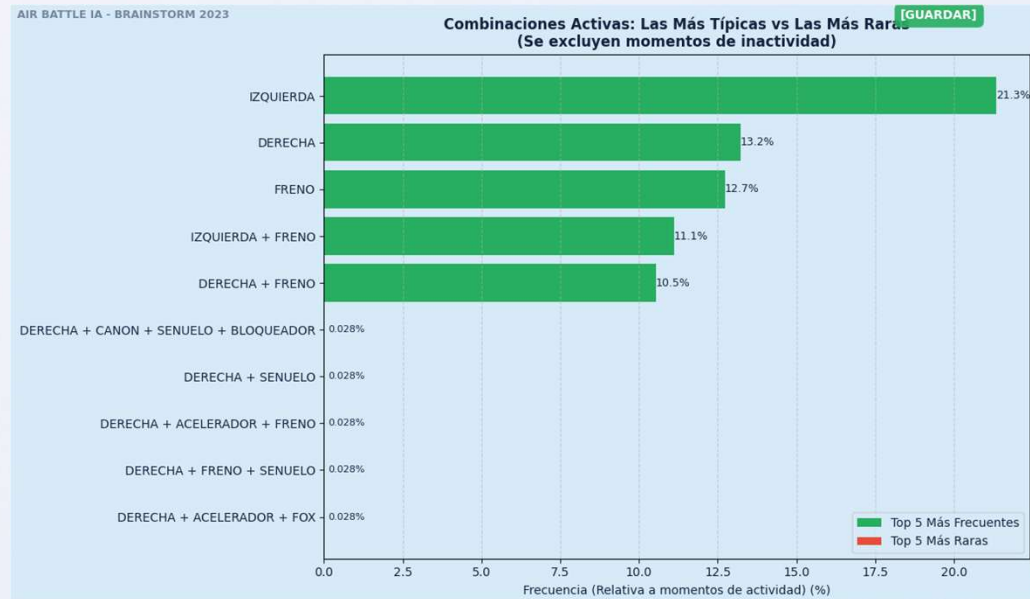
- Top 5 más frecuentes
- Top 5 más raras

Para qué se usa: detectar desbalance del "powerset".

Aunque cada etiqueta individual esté balanceada, las **combinaciones específicas** pueden ser extremadamente raras. Esas combinaciones raras son las que el modelo casi nunca aprenderá a predecir bien.

Relación con otras figuras:

Mayor nivel de detalle que la **Figura 4** (que solo cuenta *cuántos*, no *cuáles*).





Etapa 3.3: Análisis del DATASET.

Figura 6 — Frecuencia de Situaciones de Juego

Barra con la frecuencia (%) de las 5 columnas SIT_*, derivadas heurísticamente de sensores (definidas en el código, líneas 157-161):

SIT_Esquivar_Bordes

- Condición: distancia a cualquiera de los 4 bordes del mapa (SUP/INF/IZQ/DER) < 1 .
- Implica: el jugador está **al límite del área jugable** y necesita maniobrar para no salirse o chocar contra el borde.

SIT_Combate

- Condición: VISUAL_TRACK == 1 o RADAR_TRACK == 1.
- Implica: el jugador tiene a un **enemigo detectado**, ya sea visualmente o por radar. Es la condición "hay un enemigo cerca", la más general de las situaciones ofensivas.

SIT_Lanzamiento_Fox

- Condición: RADAR_TRACK == 1 o FOX_LOCK == 1 o FOX_ONAIR == 1.
- Implica: el jugador está en proceso de **preparar o ejecutar un disparo tipo "Fox"** (misil guiado) — desde el tracking por radar hasta el lock-on o el misil ya en vuelo.

SIT_Escape_Misil

- Condición: ENEMY_RADAR_TRACK == 1 o MSL_LOCK == 1 o MSL_ONAIR == 1.
- Implica: el jugador está siendo **detectado o atacado** por el enemigo (radar enemigo activo, lock enemigo sobre el jugador, o misil enemigo en vuelo).

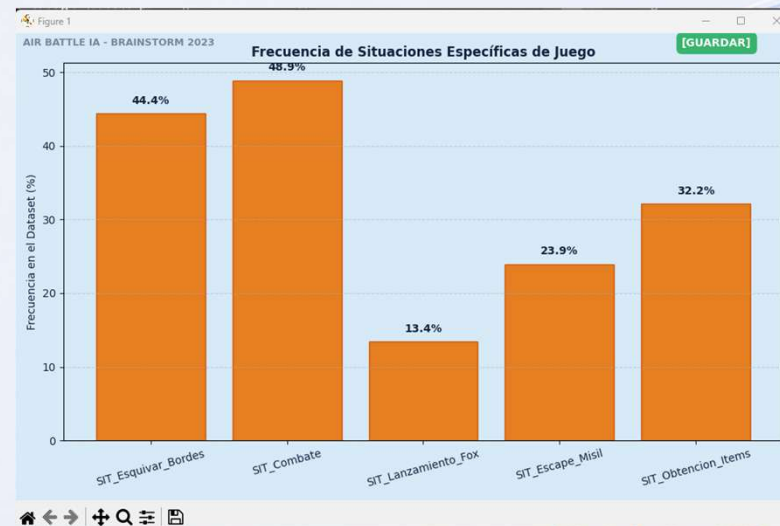
SIT_Obtencion_Items

- Condición: distancia a cualquier ítem (Combustible, Misil, Señuelo, Bloqueador) < 1 .
- Implica: el jugador está **cerca de recoger un power-up/recurso** en el mapa (recarga de combustible, munición, contramedidas, etc.).

Nota: las condiciones no son mutuamente excluyentes — un mismo frame puede activar varias SIT_* a la vez (ej. estar en combate Y cerca de un borde). Por eso las frecuencias del gráfico no suman 100%.

Para qué se usa: dar **contexto situacional** al análisis.

- Las Figuras 1-5 hablan de qué *comandos* aprieta el jugador.
- Esta figura habla de en qué *situaciones* se encuentra.
- Clave para detectar situaciones raras pero críticas (ej. baja frecuencia de SIT_Escape_Misil = pocos ejemplos para entrenar esa conducta).





Etapas 3.3: Análisis del DATASET.

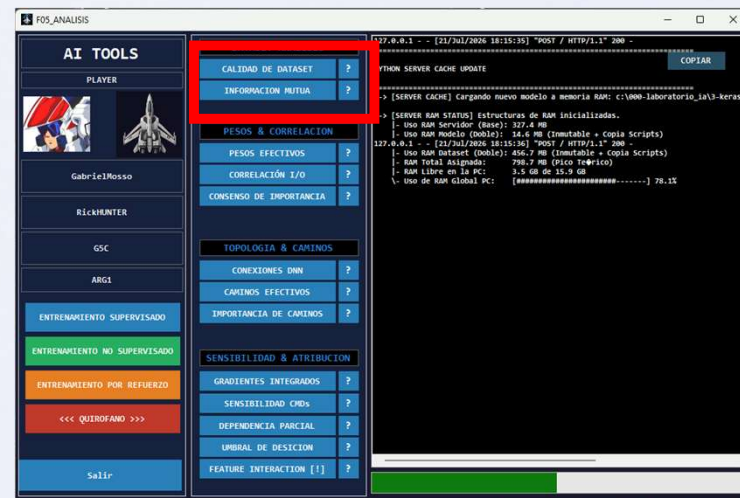
Script: CHECK_DA_01_CALIDAD_DEL_DATASET.py

Objetivo general: auditar la calidad del dataset de entrenamiento desde múltiples ángulos: balance de clases, ruido en las decisiones, estabilidad temporal y complejidad de las etiquetas.

Pregunta que responde	Figura(s)
¿Está balanceado cada comando individualmente?	1 (izq)
¿Qué comandos se usan juntos?	1 (der), 5
¿Hay etiquetas contradictorias para el mismo estado?	2
¿Cambié el estilo de juego en el tiempo?	3
¿Cuántos botones a la vez, en promedio?	4
¿Qué combinaciones exactas son raras?	5
¿En qué contexto/situación ocurre todo esto?	6

Conclusión: en conjunto arman un diagnóstico completo de calidad para dataset multilabel:

- **Balance de clases** (Fig. 1)
- **Ruido / etiquetas ambiguas** (Fig. 2)
- **Estacionariedad** (Fig. 3)
- **Complejidad de la salida** (Fig. 4-5)
- **Cobertura de escenarios** (Fig. 6)



SCRIPT >>> "CHECK_DA_01_CALIDAD_DEL_DATASET.py"

SCRIPT >>> "CHECK_DA_02_SENSOR_MUTUAL_INFORMATION.py"



Etapas 3.3: Análisis del DATASET.

Análisis de Redundancia entre Sensores (Mutual Information) — Descripción Técnica de Gráficas

Script: CHECK_DA_02_SENSOR_MUTUAL_INFORMATION.py

Objetivo Principal del Script:

Medir la **redundancia de información entre pares de sensores** (relación *input-input*), a diferencia de un análisis de "importancia de entradas" que mide relevancia *input-output* (cuánto influye cada sensor en la salida de la red).

Es la mitad complementaria de un flujo de **feature selection**:

- Un sensor puede ser muy **relevante** para la salida, pero si es muy **redundante** con otro sensor, uno de los dos podría eliminarse sin perder poder predictivo → red más simple y rápida.
- Un sensor poco relevante pero también poco redundante (independiente) puede seguir aportando información marginal útil.

Combina **tres medidas de dependencia estadística**, cada una sensible a un tipo distinto de relación entre dos sensores:

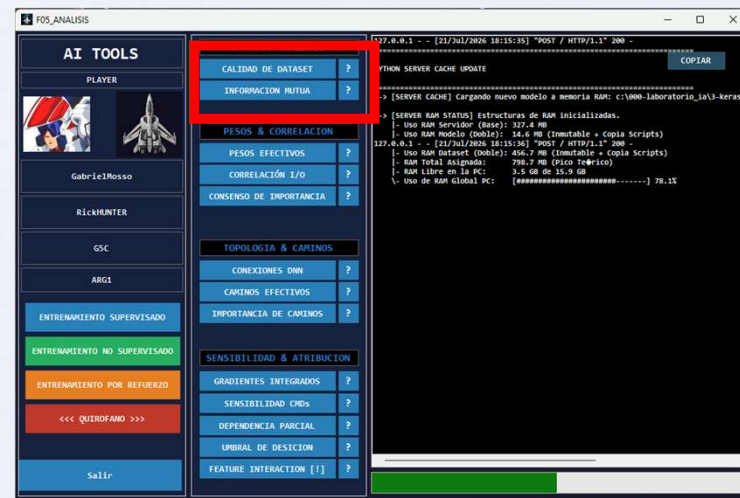
Pearson → relaciones **lineales** (con signo).

Spearman → relaciones **monótonas** (con signo), robusta a outliers y no linealidades monótonas.

Información Mutua (MI) → relaciones **generales**, incluye no lineales y no monótonas (estimador KSG de sklearn).

Calcula además un **umbral de significancia** para la MI mediante **prueba de permutación** (baraja cada sensor de forma independiente para destruir la dependencia real y mide la MI "de ruido" resultante), evitando reportar cualquier valor de MI como automáticamente relevante.

La diferencia entre MI y el máximo de |Pearson|/|Spearman| ("**brecha de no-linealidad**") permite detectar pares redundantes que un análisis de correlación clásico **no detectaría**.



SCRIPT >>> "CHECK_DA_02_SENSOR_MUTUAL_INFORMATION.py"

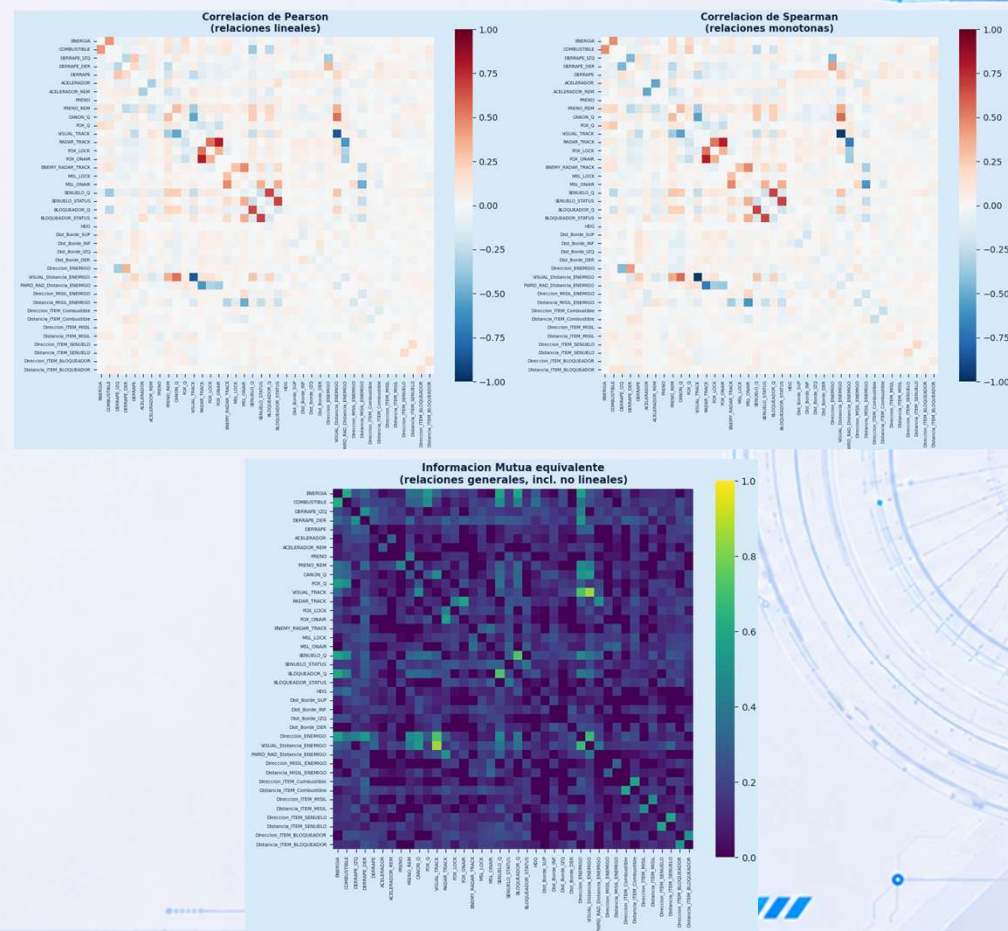
Figura 1 — Comparativa de Medidas de Dependencia

- Pearson** (RdBu_r, rango -1 a 1) → relaciones lineales.
- Spearman** (RdBu_r, rango -1 a 1) → relaciones monótonas.
- MI-equivalente** (viridis, rango 0 a 1) → relaciones generales (incluye no lineales).

Para qué se usa: dar una vista panorámica y comparar, sensor por sensor, si una relación es capturada por las 3 medidas o solo por la MI (lo cual ya insinúa no-linealidad).

Relación con otras figuras:

- Es la base visual de la **Figura 2** (que calcula la diferencia explícita entre estos paneles).
- Los valores de MI-equivalente de este heatmap alimentan directamente las **Figuras 4 y 5**.





Etapas 3.3: Análisis del DATASET.

Figura 2 — Detección de Redundancia No-Lineal

Panel izquierdo: Heatmap de "brecha no-lineal"

Calcula MI-equivalente - $\max(|\text{Pearson}|, |\text{Spearman}|)$ para cada par de sensores.

Cuanto más alto el valor, más redundancia existe entre ese par que **Pearson y Spearman no logran capturar**.

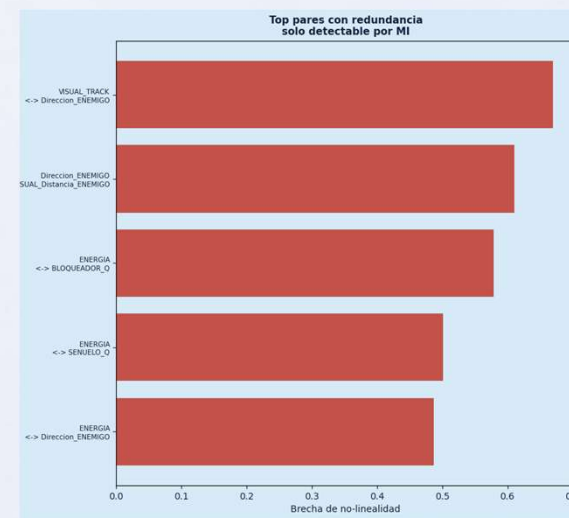
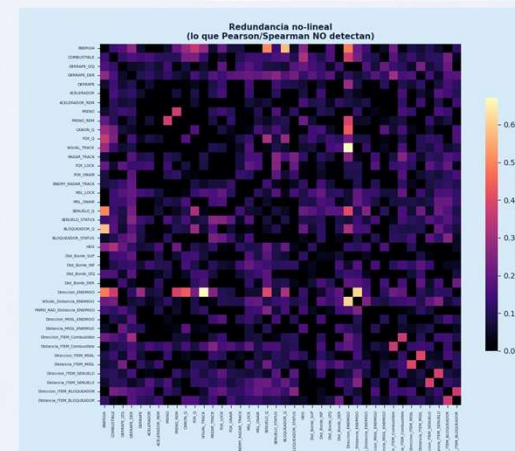
Panel derecho: Top 5 pares con mayor brecha (barh)

Lista los 5 pares de sensores con mayor redundancia "invisible" para las correlaciones clásicas.

Para qué se usa: identificar pares de sensores que **parecen independientes bajo un análisis de correlación tradicional**, pero que en realidad comparten información de forma no lineal — el tipo de redundancia que un ingeniero podría pasar por alto si solo mirara Pearson.

Relación con otras figuras:

Es un "zoom" sobre la diferencia entre los paneles 1 y 3 de la **Figura 1**.



Aristas con $MI_{eq} \geq 0.228$ (nbral de significancia = 0.0228, percentil 95 de la distribución nula)
[Haz CLICK en un nodo para resaltar sus conexiones]

Redundancia media (MI-equivalence)

0.9
0.8
0.7
0.6
0.5
0.4



Etap 3.3: Análisis del DATASET.

Figura 5 — Rankings de Redundancia e Independencia

Panel izquierdo: Top 10 pares más redundantes (barh)

Ordenado por MI-equivalente descendente.

Color **verde** = estadísticamente significativo (supera el umbral de permutación) | **gris** = no significativo.

Línea vertical roja punteada = umbral de significancia.

Panel derecho: Sensores más independientes (barh)

Top 12 sensores con **menor** MI-equivalente promedio respecto al resto (mean_mi).

Para qué se usa: dar una lista accionable y priorizada:

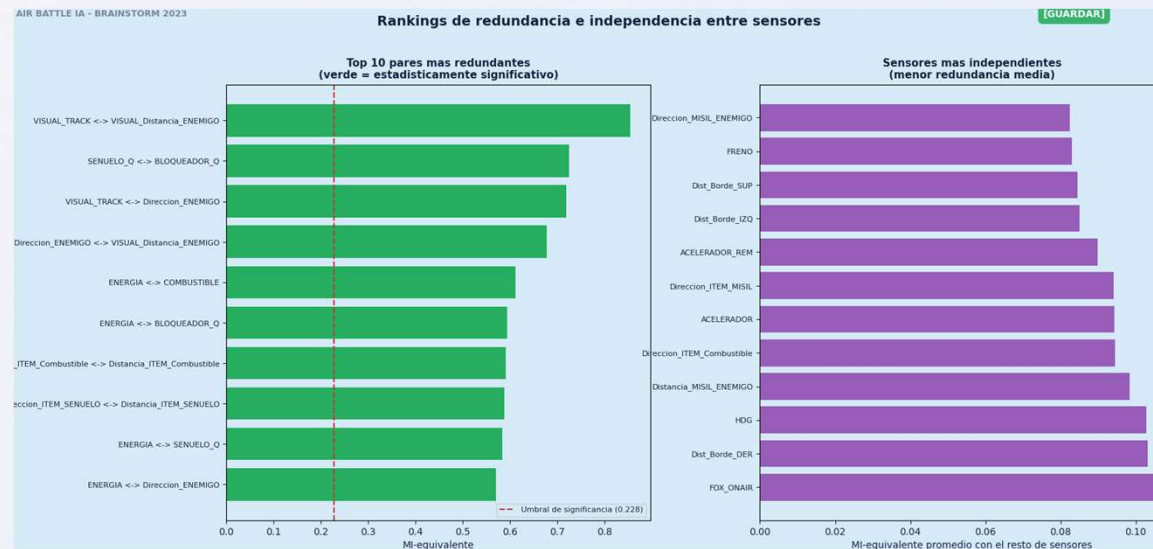
Los pares del panel izquierdo son **candidatos a eliminar uno de los dos** (redundancia alta y confirmada estadísticamente).

Los sensores del panel derecho son los que **más vale la pena conservar**, porque aportan información que ningún otro sensor replica.

Relación con otras figuras:

Resume en formato de ranking simple lo que la **Figura 4** muestra como red completa.

El "significativo/no significativo" del panel izquierdo usa el mismo umbral calculado en el Proceso 5 que define las aristas de la **Figura 4**.





Etapas 3.3: Análisis del DATASET.

Análisis de Redundancia entre Sensores (Mutual Information) — Descripción Técnica de Gráficas

Script: CHECK_DA_02_SENSOR_MUTUAL_INFORMATION.py

Objetivo Principal del Script:

Medir la **redundancia de información entre pares de sensores** (relación *input-input*), a diferencia de un análisis de "importancia de entradas" que mide relevancia *input-output* (cuánto influye cada sensor en la salida de la red).

Pregunta que responde

¿Cómo se comparan Pearson, Spearman y MI sensor por sensor?

¿Qué redundancia es invisible para correlación clásica?

¿Cuál es la estructura global de redundancia entre todos los sensores?

¿Qué pares específicos conviene podar y qué sensores conviene conservar?

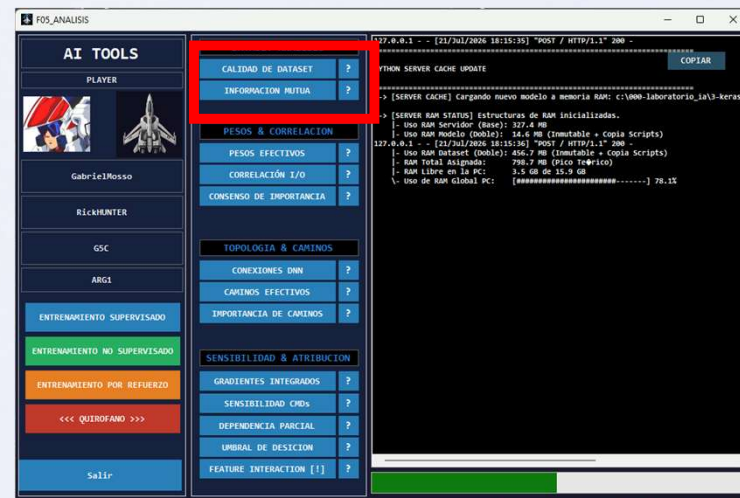
Figura(s)

1

2

4

5



SCRIPT >>> "CHECK_DA_02_SENSOR_MUTUAL_INFORMATION.py"

Conclusión: en conjunto arman un pipeline de **feature selection** basado en redundancia: comparación de métricas (1), detección de no-linealidad oculta (2), estructura relacional global (4) y recomendaciones accionables de poda/conservación (5), todo respaldado por un umbral de significancia estadística en vez de valores de MI crudos sin contexto.



Etapas 3.3: Reentrenar modelo con dataset optimizado.

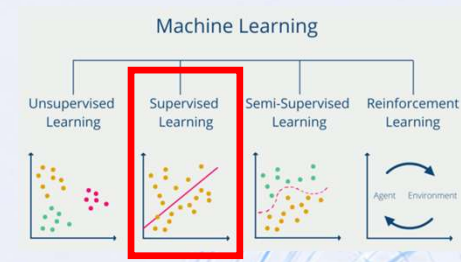
Usar el **dataset depurado y mejorado con reglas heurísticas** para volver a entrenar el modelo. El propósito es que la red neuronal aproveche **tanto la experiencia de las partidas automáticas como las correcciones impuestas por reglas humanas** para alcanzar un comportamiento más robusto y seguro en el entorno AirBattle.

Tareas específicas:

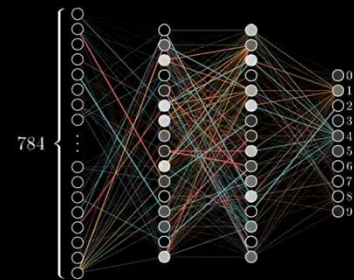
- **Cargar el nuevo dataset (Dataset_Depurado.txt):**
- **Reinicializar o cargar el modelo previo:** Cargar pesos anteriores y **seguir entrenando** para no perder el conocimiento acumulado.
- **Entrenar el modelo con el dataset modificado:** Ajustar batch_size, epochs y optimizador. Monitorear pérdida (loss) y métricas de precisión.
- **Guardar el modelo actualizado:** Exportar pesos y arquitectura en un nuevo archivo (modelo_reentrenado.h5).

Errores frecuentes en esta etapa

- **Sobrescribir el modelo previo sin respaldo:** Se pierde la comparación entre versiones (modelo inicial vs. modelo reentrenado).
- **Overfitting al dataset modificado:** El modelo memoriza las correcciones pero pierde generalización, especialmente si el dataset es pequeño.
- **Reglas mal aplicadas en 3.2 reflejadas en el modelo:** Si las reglas fueron demasiado estrictas o erróneas, el modelo aprenderá un comportamiento sesgado.
- **Olvidar probar en simulación real:** El modelo puede verse "perfecto" en métricas, pero en el juego real tomar decisiones inútiles.



Training in progress. . .



```
SCRIPT >>> "TRAIN03_AIRBATTLE_ENTRENAMIENTO.py"
```



Etapa 3.4: Evaluación de métricas del modelo

Analizar qué tanto aprendió el modelo y **cuáles entradas (inputs) tienen más impacto sobre las decisiones (outputs).**

Esto permite detectar **variables inútiles, correlaciones falsas o comportamientos inesperados** antes de lanzar otra ronda de autogeneración de partidas.

Tareas específicas:

Medir impacto de las entradas (6-ANALISIS IMPACTO.py):

- Se extraen los **pesos de la primera capa** de la red neuronal.
- Permite ver **qué inputs (ej. FUEL, STALL, ENEMY_RADAR_TRACK)** son más determinantes en la propagación hacia adelante.
- Error típico:** confundir magnitud de pesos con importancia absoluta (a veces inputs normalizados cambian la interpretación).

Analizar pesos efectivos Input-Output (8-PESOS EFECTIVOS INPUTS-OUTPUTS.py):

- Se multiplican los pesos de todas las capas para aproximar el “camino” de influencia entre cada input y cada output.
- Permite generar una **matriz de relevancia Input → Output**.
- Error típico:** interpretar correlaciones espurias como “lógica aprendida”.

Calcular correlación estadística Input-Output (7-CORRELACION INPUTS-OUTPUTS.py):

- Se miden coeficientes de correlación (Pearson o Spearman) entre las columnas de entradas y las salidas predichas.
- Diferencia con los pesos: aquí no se mide la red, sino la **relación estadística en los datos y predicciones**.
- Error típico:** usar correlación lineal en fenómenos no lineales → se pierden relaciones útiles.

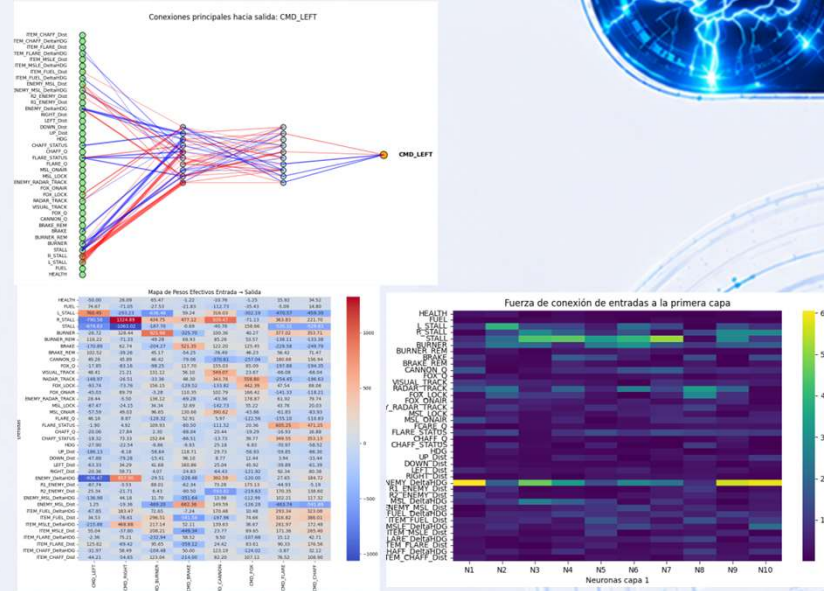
Errores frecuentes en esta etapa

Confundir métricas de entrenamiento (loss, accuracy) con análisis de pesos: Una buena accuracy no significa que el modelo esté usando los inputs correctos.

Interpretación equivocada de pesos: Pesos grandes no siempre significan más importancia (puede depender de la normalización de datos).

Omitir verificación con lógica del dominio: Se puede **aprender patrones espurios o irrelevantes** que están en los datos, aunque no tengan sentido desde el punto de vista del sistema real.

No comparar con versiones anteriores: La gracia de esta etapa es ver cómo cambió el modelo respecto al inicial y al reentrenado.



```
SCRIPT >>> "CHECK01_PESOS_EFECTIVOS_IO.py"
```

```
SCRIPT >>> "CHECK02_CORRELACION_IO.py"
```

```
SCRIPT >>> "CHECK03_GRAFICO_CONEXIONES.py"
```

```
SCRIPT >>> "CHECK04_ANALISIS_IMPACTO_INPUTS.py"
```

```
SCRIPT >>> "CHECK05_CAMINOS_EFECTIVOS.py"
```

```
SCRIPT >>> "CHECK06_CAMINOS_EFECTIVOS_IMPORT.py"
```




```
> print("Gracias !")
> print("dudas, consultas...")
> input("Presione una tecla para salir...")
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep. Argentina 786, Sunchoales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383